

Forge Delta

Exact Observable Closure and Constructive Countermodels

Universal fold verification, relational invariant synthesis,
and independently replayable intuitionistic diagnostics

Prepared for Vladimir Reshetnikov

September 15, 2026

The proposed increment. Replace a search for individually conserved expressions with an exact, goal-directed closure of polynomial observations. This proves properties of all finite input sequences and discovers the relational invariants needed to compare accumulator programs. Add finite Kripke countermodels as semantic certificates for a precisely delimited constructive proof-search diagnostic.

Executed evidence. The accompanying package contains 151 passing Python tests, 37 accepted closure certificates, 28 concrete execution counterexamples, and 8 accepted Kripke countermodels. All 73 certificates replay with Python site packages disabled. Nine bounded cases carry no conclusion.

Boundary. This is not an installed Lean tactic. No included Lean source was compiled here; 222 generated local identity obligations and three generic induction bridges are supplied as integration targets. There is no measured comparison against `grind` or any other Lean tactic.

Reviewed Forge baseline: 674521027d968d59f7b83220ed52304a85cb55e2.

Source snapshots, precise scope, results, and reproduction commands are recorded inside this article and the accompanying archive.

Abstract

Forge already proposes a certificate-producing controller above Lean’s local reasoning engines. Repeating that architecture would not advance the project. This article instead supplies two operational additions. The first computes the least family of polynomial vector spaces containing the demanded observations and closed under coefficientwise pullback along control-flow edges. It emits finite matrix identities that prove unbounded, arbitrary-input executions by induction. For updates affine in state, though polynomial in fresh inputs, the procedure is a decision method in exact arithmetic: a space of dimension at most $|Q|\binom{r+D}{D}$ suffices for degree- D goals. A failed initial obligation has a constructive execution witness, not merely a failed invariant candidate. The second addition proposes and independently checks finite Kripke countermodels. It establishes nonderivability in a named intuitionistic object logic, never the negation of an arbitrary Lean proposition. We give complete mathematical arguments, algorithms, certificate schemas, source-to-Lean integration plans, implementation constraints, and reproducible synthetic experiments. The conceptual ingredients are established mathematics; the contribution is their concrete, incremental application to Forge with a tested search/checker separation.

Contents

1	The actual increment, not another Forge architecture	4
1.1	What the reviewed baseline already contains	4
1.2	Two recommendations	4
1.3	Relation to established work	5
1.4	Evidence labels used throughout	5
2	The machine fragment and the proof obligation	5
2.1	State, control, and fresh inputs	5
2.2	Why this is a useful target for Lean automation	6
2.3	Arbitrary starting parameters without a new search mechanism	6
2.4	What is deliberately outside the first fragment	6
2.5	The small example that motivates the construction	7
3	Goal-directed observable closure	7
3.1	Linear operators hidden inside polynomial updates	7
3.2	The least closed family	7
3.3	Exact dependence without losing useful provenance	8
3.4	The positive certificate	8
3.5	Search and replay really use different arithmetic paths	9
4	Soundness, completeness, and a constructive negative result	9
4.1	Soundness for arbitrary polynomial updates	9
4.2	What the constructed space means	10
4.3	Constructing a counterexample instead of merely rejecting a basis	10
4.4	A complete fragment with a finite dimension bound	11

4.5	Nonlinear success and nonlinear failure are both informative	12
4.6	Cost and specialization	12
5	Worked proof plans and genuinely distinguishing cases	12
5.1	Delayed equality: both the readable basis and the emitted basis	12
5.2	Different Fibonacci accumulator presentations	13
5.3	A quadratic identity with arbitrary input magnitudes	13
5.4	A finite-control invariant	13
5.5	A simultaneous-update regression	14
5.6	The finite-sample trap	14
5.7	A family with known positive and negative answers	14
6	Finite Kripke certificates for constructive diagnostics	14
6.1	The question this worker actually answers	14
6.2	The certificate and its forcing semantics	15
6.3	Why classical tautologies are essential negative tests	16
6.4	A branching model catches what chains miss	16
6.5	The executable bounded proposer	16
6.6	A concrete scalable SAT backend, proposed but not executed	17
7	Concrete Lean, Forge, Leant, and Djex integration	17
7.1	Do not invent a new outer controller	17
7.2	The smallest useful source-to-machine reifier	18
7.3	The initial Lean proof path: source reconstruction	18
7.4	A reflected checker is the next optimization, not a prerequisite	19
7.5	Universal behavioral assertions in Leant	19
7.6	Source-level quantifiers and arithmetic domains	19
7.7	A precise Kripke-to-Lean target	20
8	Executed experiments and their limits	20
8.1	Protocol and retained artifacts	20
8.2	Mechanism ablation, not a tactic comparison	21
8.3	Adversarial and differential checks	21
8.4	Timing observations are intentionally secondary	21
8.5	A failed attempt retained, rather than erased	22
8.6	What these experiments establish, and what they do not	22
9	Implementation plan, libraries, and acceptance gates	22
9.1	Concrete library choices	22

9.2	Suggested module additions	23
9.3	Gate 1: the first actual Lean milestone	23
9.4	Gate 2: a real reifier with exact-source controls	24
9.5	Gate 3: universal behavioral acceptance in Leant	24
9.6	Gate 4: IPC diagnostics that cannot refute classical Lean	24
9.7	Gate 5: evaluate real capability, then optimize	24
9.8	Optimizations that preserve the certificate language	25
10	Research extensions and final recommendation	25
10.1	Where the next mathematical work should go	25
10.2	What should be merged first	26
10.3	The substantive change in capability	26
A	Certificate formats and executable entry points	26
A.1	Canonical rational polynomials	26
A.2	Machine and positive certificate	27
A.3	Execution certificate	27
A.4	IPC problem and countermodel	27
A.5	Reproduce the experiment	28
A.6	Build the article and attempt the Lean gates	28
B	Source audit, scope of originality, and artifact ledger	29
B.1	Pinned repository context	29
B.2	Contribution and reuse ledger	29
B.3	Archive contents	30
B.4	Remaining acceptance obligations	30

1 The actual increment, not another Forge architecture

1.1 What the reviewed baseline already contains

The baseline is Forge commit 674521027d968d59f7b83220ed52304a85cb55e2, particularly the merged article’s structural-search section, provenance appendix, status ledger, and review of Leant and Djex. It already covers induction planning, accumulator generalization, demand-directed theorem use, polynomial recurrence synthesis, algebraic certificates, and a carefully delimited acceptance boundary [1, 2, 3, 4]. These are dependencies of this proposal, not its contributions.

The relevant unfinished step is narrower and more consequential. The implemented state-machine invariant certificate described in the merged article requires $I(F(x)) = I(x)$. Its prospective generalization allows multiple invariants and polynomial multipliers, but correctly warns that discovering both at once is a bilinear problem. The first contribution below supplies a different search organization: demand a *closed space of observations*, discover it by exact linear dependence, and then prove that its generators all vanish on the reachable states. No simultaneous unknown invariant/multiplier system is solved.

The second relevant gap is already named in the repository: negative results from an intuitionistic inhabitation decision procedure lack a separately checkable certificate. The proposal here does not claim that identifying this gap is new. It provides an implemented *semantic alternative* to logging an entire unsuccessful LJT search: a finite Kripke structure and a small forcing checker.

Existing ingredient	Increment in this package	Resulting capability
Conserved polynomial invariant	Control-indexed coefficient-pullback closure	Mutually supporting relational invariants, discovered rather than supplied
Polynomial recurrence goals	Fresh symbolic payloads at every edge	All finite words over an infinite rational input alphabet
Typed candidate plus bounded behavioral tests	Product-machine proof obligation	Universal fold equivalence in an explicit fragment
Suggested failed-search certificate	Finite Kripke model and independent forcing	Replayable IPC nonderivability diagnostics
Uncompiled certificate sources	222 generated local identity obligations	A concrete next compiler gate, not a claim of Lean acceptance

Table 1: The delta relative to the reviewed merged design. The last row is an integration artifact, not a new logical method.

1.2 Two recommendations

Implement the observable-closure worker first. It addresses a positive proof capability directly: statements about arbitrary finite executions, streaming algorithms, accumulator programs, and equivalence of two folds. Its certificate needs only exact polynomial identities plus induction, and it can reuse existing Lean arithmetic reconstruction.

Implement the Kripke worker as a diagnostic lane. It is valuable when a user asks whether constructive propositional reasoning is sufficient. It must not close the original goal with its negation. Its strongest appropriate output is a checked theorem that the corresponding *object-logic sequent* has no derivation.

These are not substitutes for each other. A rational execution counterexample can refute a particular universally quantified machine property. An IPC countermodel instead classifies a restricted proof language. An interface that uses one generic “refuted” label for both would erase the most important semantic distinction in the proposal.

1.3 Relation to established work

The algebraic worker belongs to the tradition of exact invariant analysis, including Karr’s affine relations, and to the linear-algebraic treatment of observables under composition. Finite invariant spaces of Koopman observables are an established perspective on nonlinear dynamics; this proposal uses exact polynomial pullbacks for discrete, symbolic-input proof obligations, not numerically learned approximate observables [8, 9]. Register automata with linear arithmetic supply closely related algorithmic context [10]. Neither invariant subspaces nor rational-machine verification is claimed to originate here.

Kripke semantics is likewise established. Lean formalizations of intuitionistic propositional logic already provide soundness and completeness infrastructure [11]. Mathlib’s `itauto` already implements a complete positive intuitionistic propositional procedure for its supported syntax [13]. The addition is a reusable negative *certificate boundary*, not a new claim to solve positive propositional tautologies.

1.4 Evidence labels used throughout

Proved mathematically means a proof is given in this article. *Executed* means the supplied code actually ran and has retained results. *Generated, uncompiled* means Lean text was emitted but no Lean compiler checked it. *Proposed* means an implementation plan, not completed code. These labels must not be collapsed into a single word such as “verified.” The supplied Python checker is an executable implementation of the mathematics; it is not itself formally verified.

2 The machine fragment and the proof obligation

2.1 State, control, and fresh inputs

Let Q be a finite set of control nodes. At each node the program has r rational registers $x = (x_1, \dots, x_r)$. An edge $e : q \rightarrow q'$ carries a *simultaneous* polynomial update

$$F_e : \mathbb{Q}^r \times \mathbb{Q}^m \longrightarrow \mathbb{Q}^r, \quad F_e(x, u) \in \mathbb{Q}[x_1, \dots, x_r, u_1, \dots, u_m]^r.$$

An execution begins at a specified (q_0, x_0) . At every step it chooses an edge leaving the current node and a fresh arbitrary rational payload $u \in \mathbb{Q}^m$. It then moves to $(q', F_e(x, u))$. The zero-step execution is included.

The edge choice is unrestricted among outgoing edges. It can represent an input constructor or tag. The implemented fragment has *no numerical edge guards*. In particular, an edge labelled “positive” is not assumed to have a positive payload: labels are descriptive strings, not formulas.

For each control node q the demanded outputs are a finite list

$$G_q = (g_{q,1}, \dots, g_{q,t_q}), \quad g_{q,j} \in \mathbb{Q}[x].$$

The task is to establish

$$\forall(q, x) \text{ reachable from } (q_0, x_0), \bigwedge_{j=1}^{t_q} g_{q,j}(x) = 0. \quad (1)$$

An empty list at a node imposes no output condition there. It does not exempt that node from the backward propagation needed to prove later outputs.

2.2 Why this is a useful target for Lean automation

A fold with a fixed-dimensional accumulator has exactly this shape. Suppose

$$f(xs) = o_f(\text{foldl}(F, s_f, xs)), \quad h(xs) = o_h(\text{foldl}(H, s_h, xs)).$$

To prove $\forall xs, f(xs) = h(xs)$, take the product accumulator (s, t) , update both sides on the *same* input, and demand $o_f(s) - o_h(t) = 0$. Different implementation states are allowed: a two-register candidate can be compared with a three-register reference by using five registers in the product. The relation between those registers need not be supplied by the user.

An invariant of the product is not merely an identity of program syntax. It is a relational assertion about all states reached by the two executions. This is what permits different recurrences, coordinate systems, and accumulator representations to be related.

The product construction is valid only when both sides consume the same input word with the same step alignment. Comparing a one-pass fold to a program that conditionally consumes two elements needs a richer simulation relation; silently pairing their updates would prove the wrong theorem.

2.3 Arbitrary starting parameters without a new search mechanism

A universally quantified starting parameter θ can be modeled by a special initial control node with no demanded output and a single initialization edge. Its payload is θ , and its update installs the desired polynomial initial state $H(\theta)$. The edge goes to the ordinary execution node, and no edge returns to the initialization node.

For example, the universal claim that $b = 7a$ remains true after increments $(a, b) \mapsto (a + u, b + 7u)$ can start at $(a, b) = (\theta, 7\theta)$. The implemented case `symbolic_initial_parameter` encodes precisely this construction. It is not finite sampling of θ . The source bridge must identify the distinguished first edge, and input tuples can be padded with unused coordinates so that the common payload dimension remains fixed.

2.4 What is deliberately outside the first fragment

There is no integer division, truncated natural subtraction, floating-point rounding, finite-field interpretation, variable-length register vector, or uninterpreted state operation. Numerical guards, probabilistic choices, and unbounded heap structures are not represented. Polynomial updates may be nonlinear in state, but then the closure algorithm can fail to terminate without a resource cap.

Over-approximating a guarded machine by dropping its guards has a useful *one-sided* meaning: a positive invariant of all the additional executions remains valid for the guarded program. A counterexample to that over-approximation need not be executable in the original program. The initial integration should therefore refuse guarded translations rather than allow the negative lane to acquire unintended authority.

2.5 The small example that motivates the construction

Consider four registers initially zero and the update

$$(a, b, c, d) \mapsto (a + b, b + u, c + d, d + u). \quad (2)$$

The demand is $a - c = 0$. It is true on every reachable state. Nevertheless,

$$(a - c) \circ F = (a - c) + (b - d) \neq a - c$$

as a polynomial on arbitrary states. The target is not conserved. The missing fact is $b - d = 0$, and it is conserved. Discovering the two-dimensional space spanned by $a - c$ and $b - d$ solves the problem without a general invariant ansatz or a guessed coefficient bound.

3 Goal-directed observable closure

3.1 Linear operators hidden inside polynomial updates

For an edge $e : q \rightarrow q'$ and polynomial observation $p \in \mathbb{Q}[x]$, expand

$$p(F_e(x, u)) = \sum_{\alpha \in \mathbb{N}^m} u^\alpha T_{e,\alpha}(p)(x). \quad (3)$$

Only finitely many coefficients are nonzero. For fixed e and α , $T_{e,\alpha}$ is a *linear map of rational vector spaces*, even when F_e is nonlinear in x . Composition with a fixed map and extraction of a coefficient both commute with rational linear combinations.

This observation separates two issues often conflated in invariant synthesis. The degree of $p(F_e)$ may grow, creating more monomials. But testing whether its coefficients belong to a space already discovered is still exact linear algebra. The procedure does not make an unknown polynomial multiplier multiply an unknown candidate invariant.

3.2 The least closed family

We seek vector spaces $W_q \subseteq \mathbb{Q}[x]$ satisfying

$$g_{q,j} \in W_q \quad \text{for every demanded goal,} \quad (4)$$

$$T_{e,\alpha}(W_{q'}) \subseteq W_q \quad \text{for every } e : q \rightarrow q' \text{ and every } \alpha. \quad (5)$$

There is a least such family: intersect all families satisfying these conditions, or equivalently take the span of all iterated coefficient pullbacks of the goals. The search constructs bases P_q for that family whenever the process stabilizes.

Backward propagation is essential. A newly discovered observation at the *destination* of an edge is pulled back to the *source*. Moving it in the other direction is not a sound invariant-discovery algorithm.

```

for each control node q:
  basis[q] := empty exact rational span
agenda := empty queue
insert every requested goal into its node's span

```

```

while agenda is nonempty:
  (destination, p) := pop_front(agenda)
  if destination is initial and p(initial_state) != 0:
    return a reconstructed, independently checked execution trace
  for each incoming edge e : source -> destination:
    expanded := simultaneous_substitution(p, update[e])
    for each nonzero payload coefficient c(x) of expanded:
      if c is not in span(basis[source]):
        retain c itself, with its parent/edge provenance
        insert c into the exact span and enqueue it

emit goal-coordinate and edge-coordinate identities
accept only after independent replay of every identity

```

There are separate limits for state degree, basis size, coefficient obligations, polynomial terms, and counterexample-grid enumeration. A cutoff returns UNKNOWN. The bound on worklist actions is not a wall-clock preemption mechanism; a large symbolic expansion can itself be expensive.

3.3 Exact dependence without losing useful provenance

The implementation retains each *original* independent polynomial rather than replacing it by its reduced echelon residual. This choice is intentional. An original polynomial has a simple origin: either a demanded goal, or one coefficient of the pullback of a previously retained polynomial. That origin will later construct a counterexample trace.

A separate echelon structure stores normalized pivot rows and their coordinates in the retained original generators. To reduce a new polynomial p , eliminate its pivot monomials, accumulating coefficients c_i , until

$$p = \sum_i c_i P_i + r.$$

If $r = 0$, p is already in the space. Otherwise retain p and insert $r/\text{lc}(r)$ as a pivot row. Its coordinates in the enlarged original basis are

$$\left(-\frac{c_1}{\text{lc}(r)}, \dots, -\frac{c_k}{\text{lc}(r)}, \frac{1}{\text{lc}(r)} \right).$$

Exact rational arithmetic makes this a decision, not a tolerance-based rank heuristic. Every previous coordinate row is padded with a zero when a generator is added. Monomial order is fixed, so certificate output is deterministic for a fixed input and traversal order.

3.4 The positive certificate

Write P_q as a column vector of its retained generators. Search returns rational matrices C_q and polynomial matrices $M_e(u)$ such that

$$G_q(x) = C_q P_q(x), \tag{6}$$

$$P_{q'}(F_e(x, u)) = M_e(u) P_q(x) \tag{7}$$

$$P_{q_0}(x_0) = 0. \tag{8}$$

The matrix M_e has $\dim W_{q'}$ rows and $\dim W_q$ columns; reversing these dimensions is an easy implementation error. A zero-dimensional source space is valid only when each relevant pulled-back observation is identically zero. A zero-dimensional destination has no preservation rows.

The checker does not need to verify that the generators are independent, that the family is least, or that the search followed this algorithm. Those properties concern discovery, completeness, and cost. Soundness requires only (6)–(8), with complete coverage of the *externally supplied* problem’s edges and goals.

3.5 Search and replay really use different arithmetic paths

The search uses SymPy expressions, Poly over QQ, simultaneous `xreplace`, coefficient extraction, and a custom exact span manager. The checker uses canonical sparse dictionaries with `fractions.Fraction`, explicit addition and convolution, and independently implemented simultaneous substitution. It does not import SymPy or ask the search to normalize an identity for it.

The two sides still share the problem schema and Python runtime. The search also calls the schema validator before starting. Thus “independent” here means independent algebraic replay, not two verified implementations with independent foundations. The distinction is tested by replaying the stored corpus under `python -S`, which disables site-package imports.

4 Soundness, completeness, and a constructive negative result

4.1 Soundness for arbitrary polynomial updates

Theorem 4.1 (Finite closure certificate). *For the machine of Section 2, suppose the polynomial identities (6)–(8) hold. Then every reachable state satisfies all its demanded output equalities.*

Proof. Induct on the length of an execution. Equation (8) proves that all initial generators vanish. Suppose $P_q(x) = 0$ and an edge with payload u moves the state to $F_e(x, u)$ at q' . Evaluating (7) at this particular x, u yields $P_{q'}(F_e(x, u)) = M_e(u)0 = 0$. Thus the generators vanish after every finite execution. At its endpoint, (6) gives $G_q(x) = C_q 0 = 0$. No assumption that the update is affine, invertible, or terminating beyond the finite input word was used. $\square$

The quantifier over input words is unbounded. The certificate proves all word lengths at once; it is not an extrapolation from a bounded execution trace. The argument also explains why matrices need not be nonnegative: these are zero equalities, not order inequalities.

Corollary 4.2 (Relational fold verification). *Suppose a faithful product-machine translation of two folds has output goals $o_f(s) - o_h(t) = 0$ and admits a closure certificate. Then their outputs agree on every finite input list in the translated input domain.*

Proof. A simultaneous step of the product updates each component on the same payload. Induction on the input list identifies the product’s endpoint with the pair of fold endpoints. Apply Theorem 4.1 to its output difference. $\square$

4.2 What the constructed space means

Lemma 4.3 (Path-coefficient characterization). *For a control node q , the least space W_q is spanned by the coefficient polynomials of all expressions obtained by pulling a demanded output backward along a finite path beginning at q . Every time step uses its own fresh tuple of payload indeterminates.*

Proof. Length-zero paths contribute precisely the requested goals. Extending a path backward by one edge first substitutes that edge's update, then extracts coefficients in its fresh inputs. Coefficient extraction for the other time steps commutes with this operation because their variables are distinct. Consequently the span of all path coefficients contains the goals and is closed under every $T_{e,\alpha}$. Conversely, repeated closure applied to a goal produces only such coefficients or their rational linear combinations. Every closed family containing the goals contains them all. This proves both inclusions and leastness. $\square$

Leastness is a vector-space statement. It does not say this is the smallest inductive invariant as a set of states, or a basis of the ideal of all reachable polynomial equalities. The distinction matters for nonlinear updates.

4.3 Constructing a counterexample instead of merely rejecting a basis

Lemma 4.4 (A finite separating grid). *Let $f \in \mathbb{Q}[u_1, \dots, u_m]$ be nonzero and let d_i be its degree in u_i . There is a point in $\prod_i \{0, 1, \dots, d_i\}$ at which f is nonzero. For $m = 0$, the grid consists of the unique empty tuple.*

Proof. For one variable, a nonzero degree- d polynomial has at most d roots. For several variables, write f as a polynomial in the last variable. If it vanished on the whole grid, then at every choice of the earlier coordinates all its coefficients in the last variable would vanish. Induction on the number of variables would make every coefficient polynomial zero, contrary to $f \neq 0$. $\square$

Suppose a retained polynomial $p \in P_{q_0}$ has $p(x_0) \neq 0$. If its origin is a requested goal, the empty trace already refutes the property. Otherwise its origin records an edge $e : q_0 \rightarrow q'$ and an older polynomial $h \in P_{q'}$ for which p is one payload coefficient of $h(F_e(x, u))$. At the concrete current state x_0 , that coefficient is nonzero. The polynomial $u \mapsto h(F_e(x_0, u))$ is therefore nonzero. Lemma 4.4 supplies a concrete payload u_0 making it nonzero. Execute the edge and repeat with h .

Each parent was retained before its child. Following parents must therefore terminate at a demanded goal. The resulting certificate is a finite sequence of original edge indices and rational payload tuples, together with the index of the final output that is nonzero. A separate checker executes the original updates and verifies the final discrepancy. It does not need the basis, provenance graph, or root-bound argument.

Proposition 4.5 (Negative evidence from a retained origin). *With unrestricted rational payloads and complete original-generator provenance, a nonvanishing generator at the initial state yields a genuine finite execution counterexample. A budget cutoff while constructing its payloads justifies only UNKNOWN, not an accepted counterexample.*

Proof. At each selected step the preceding paragraph preserves the assertion that the current retained observation is nonzero. The provenance order is strictly decreasing. Its last node is an original demanded goal, and all traversed edges start at the current control node. The constructed

trace is therefore an execution whose endpoint violates that goal. Independent execution replay checks this final claim directly. $\square$

4.4 A complete fragment with a finite dimension bound

Definition 4.6 (State-affine update). An update is state-affine when

$$F_e(x, u) = A_e(u)x + b_e(u),$$

where the entries of A_e and b_e are rational polynomials in the fresh payloads. It need not be affine in the payloads.

Theorem 4.7 (Exact decision in the state-affine fragment). *Assume all updates are state-affine and every demanded output has state degree at most D . In exact arithmetic, with no resource truncation, observable closure stabilizes after at most*

$$B = |Q| \binom{r + D}{D}$$

independent generator insertions. The procedure either returns a positive certificate for (1) or returns a concrete execution counterexample.

Proof. Substitution of state-affine expressions into a degree- D observation does not increase its degree in x . Extracting coefficients in u cannot increase that degree either. All spaces therefore remain in $V_D = \{p \in \mathbb{Q}[x] : \deg p \leq D\}$, whose monomial basis has cardinality $N = \binom{r+D}{D}$. Each insertion strictly increases the dimension of one node's space. There are at most $|Q|N$ such increases, each generating finitely many coefficient obligations. The finite worklist eventually empties unless an initial generator is found nonzero.

If an initial generator is nonzero, provenance reconstruction gives a counterexample. Otherwise all initial generators vanish. At stabilization, exact linear dependence supplies the matrices in (6) and (7); hence Theorem 4.1 proves the property. Equivalently, Lemma 4.3 shows that a valid property forces every path-coefficient generator to vanish initially, since a polynomial vanishing on every rational payload assignment has zero coefficients. $\square$

Corollary 4.8 (A short counterexample word). *If a property in Theorem 4.7's fragment is false, the retained origin construction can produce a counterexample with at most $B - 1$ edges. This bounds the word length, not the bit size of its rational states.*

Proof. A nonzero retained initial generator has an ancestry chain ending in an original goal. Each edge on that chain moves to a strictly earlier retained generator. At most B generators can be retained in the whole control graph, so such a chain contains at most $B - 1$ coefficient steps. $\square$

The code enforces practical degree and basis bounds. The mathematical theorem is not a promise that those defaults are large enough for every legal input. For example, if B exceeds the configured per-node capacity, a valid problem can still produce UNKNOWN.

4.5 Nonlinear success and nonlinear failure are both informative

For $s' = s + u$ and $t' = t + 3s^2u + 3su^2 + u^3$, the degree-three observation $t - s^3$ is conserved. The nonlinear-state fragment succeeds immediately. But for $x' = x^2$, $x_0 = 0$, and goal $x = 0$, closure generates

$$x, x^2, x^4, x^8, \dots$$

which are linearly independent as polynomials. The property is true, yet the implemented degree cap correctly returns UNKNOWN.

The ideal generated by x would handle this last example because $x^2 = x \cdot x$. That observation points to a complementary use of Forge’s existing polynomial-multiplier machinery, not an excuse to describe vector-space closure as a general invariant decision procedure.

4.6 Cost and specialization

Let $N = \binom{r+D}{D}$, and suppose every pullback has at most U distinct payload monomials. Each edge is processed once for every retained generator at its destination, giving at most EN pullbacks and ENU coefficient candidates for E edges. A crude dense echelon bound is $O(EUN^3)$ rational operations, plus the cost of polynomial expansion. Dense space and coordinate storage requires $O(|Q|N^2)$ coefficients. Coefficient bit growth is not included in those arithmetic-operation counts.

The advantage of demand direction is that the actual dimensions k_q can be much smaller than N . The delayed example uses two generators out of five possible affine monomials, and the square-of-sum example uses one out of six quadratic monomials. These are exact dimension comparisons, not claims of a measured runtime advantage over a full-lifting implementation.

5 Worked proof plans and genuinely distinguishing cases

5.1 Delayed equality: both the readable basis and the emitted basis

For update (2), a convenient human basis is

$$H = \begin{pmatrix} a - c \\ b - d \end{pmatrix}, \quad H(F(x, u)) = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} H(x).$$

Both components vanish initially. This one matrix identity proves $a = c$ after arbitrarily many steps. It explains why merely checking that the target itself is conserved is insufficient.

The program retains original pullbacks, so its actual stored basis is

$$P = \begin{pmatrix} a - c \\ a + b - c - d \end{pmatrix}, \quad P(F(x, u)) = \begin{pmatrix} 0 & 1 \\ -1 & 2 \end{pmatrix} P(x).$$

These bases span the same space, but the second is the one in `results/corpus.json`. Showing the first without this qualification would misdescribe the experiment.

Now change the last update to $d' = d + u + 1$. Start at zero and feed two zero payloads. The first state is $(0, 0, 0, 1)$, so the original demand still holds. The second is $(0, 0, 1, 2)$, so $a - c = -1$. The prototype emits exactly this length-two trace. A one-step test at the initial state would miss the error.

5.2 Different Fibonacci accumulator presentations

Compare

$$(a, b)' = (b, a + b), \quad (p, q)' = (p + q, p)$$

starting at $(a, b, p, q) = (0, 1, 1, 0)$, with output demand $b - p = 0$. The additional relation $a - q = 0$ is needed. In the emitted basis

$$P_1 = b - p, \quad P_2 = a + b - p - q,$$

the transition matrix is

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

This is not the additive-polynomial recurrence fragment used to synthesize power sums: the outputs have Fibonacci growth. The proof comes from a finite relation between the two state representations, not from guessing a polynomial closed form for either output.

5.3 A quadratic identity with arbitrary input magnitudes

Let

$$s' = s + u, \quad t' = t + 2su + u^2, \quad (s_0, t_0) = (0, 0).$$

The goal is $t - s^2 = 0$. The entire closure has one generator because

$$(t + 2su + u^2) - (s + u)^2 = t - s^2.$$

There is no bound on input length or magnitude. The update is affine in state (s, t) with coefficients depending polynomially on the payload; the theorem of Section 4 applies with $D = 2$.

The case `two_payload_square` replaces u by uv . The same one-generator certificate proves the identity for all lists of rational pairs. This checks that the implementation really extracts coefficients in multiple fresh inputs rather than substituting a single test value or confusing state degree with total degree.

5.4 A finite-control invariant

Use two control nodes, even and odd, and transitions

$$\begin{aligned} \text{even} &\rightarrow \text{odd} : (s, t) \mapsto (s + u, u - t), \\ \text{odd} &\rightarrow \text{even} : (s, t) \mapsto (s - u, u - t). \end{aligned}$$

Start at the even node with $(s, t) = (0, 0)$. Demand $s + t = 0$ at the even node and $s - t = 0$ at the odd node. Each destination goal pulls back to its source goal:

$$(s + u) - (u - t) = s + t, \quad (s - u) + (u - t) = s - t.$$

The certificate has a one-dimensional basis at each node. A single unqualified statement $s = t$ would be wrong on half the control states. This is why control identity is part of the invariant semantics, not just an optimization key.

5.5 A simultaneous-update regression

The update $(a, b) \mapsto (b, a)$ with initial $(1, 2)$ preserves $a + b = 3$. Sequentially overwriting the first register before computing the second would instead produce $(2, 2)$ and break the invariant. The search uses simultaneous substitution; the replay checker evaluates every right-hand side against the same old state. A dedicated regression case and random substitution comparisons exercise this distinction.

5.6 The finite-sample trap

Consider

$$x' = x + \prod_{i=0}^6 (u - i), \quad x_0 = 0, \quad \text{demand } x = 0.$$

Every one-step sample at $u = 0, 1, \dots, 6$ passes. Repeating only those payloads for any length would also pass. The identity is nevertheless false. Coefficient extraction discovers a nonzero constant observation at the initial state, and provenance reconstruction chooses $u = 7$. The actual endpoint is $x = 7! = 5040$, so the execution checker rejects the universal equality.

The use of a small grid in the *counterexample constructor* is not the same mistake as finite-sample verification. The nonzero polynomial has already been established by exact coefficients; the grid is a proved way of finding a witness to its nonzeroness.

5.7 A family with known positive and negative answers

The synthetic conjugacy family begins with an affine machine

$$x' = Ax + Bu + c.$$

Choose an invertible rational matrix U and offset v , and express the same machine in coordinates $y = Ux + v$:

$$y' = U(AU^{-1}(y - v) + Bu + c) + v.$$

The product starts with this relation satisfied. Only its *first* coordinate is supplied as the output goal, so closure must discover any other relations needed by the selected matrices. The 24 positive cases use two or three registers per side and one or two input tags. Their 24 negative partners add 1 to the first transformed register update, creating a checked violation.

This generator gives known answers and variation in coefficients, basis coordinates, and tagged transitions. It is not independent evidence that the method handles arbitrary real-world Lean functions. Because the family was constructed from the intended algebraic structure, it is a mechanism test, not a held-out theorem benchmark.

6 Finite Kripke certificates for constructive diagnostics

6.1 The question this worker actually answers

The input is a sequent $\Gamma \vdash_{\text{IPC}} G$ in a named intuitionistic propositional object calculus. Its formulas contain atoms, $\perp$, conjunction, disjunction, and implication. A successful negative result means:

there is no derivation of G from Γ in that calculus.

It does *not* mean that Lean can prove $\neg G$, that no Lean term of G exists, or that classical reasoning cannot prove G . It says nothing about additional domain-specific hypotheses that were omitted from the object sequent.

This separation is indispensable for integration with Djex. Its formula representation distinguishes ordinary atoms, opaque source-type atoms, and dictionary atoms. The object checker can validate the propositional structure once it is supplied, but it does not establish that erasing an arbitrary Lean type or typeclass environment preserves all possible inhabitants [7]. The initial implementation should display an IPC countermodel without promoting it to unrestricted source-level uninhabitation.

6.2 The certificate and its forcing semantics

A finite model consists of worlds $\{0, \dots, n-1\}$, a root w_0 , a partial order $\leq$, and one Boolean valuation per atom. The root must be least. Atomic truth is persistent: if $w \leq v$ and an atom is true at w , it is true at v .

Forcing is defined recursively:

$$\begin{aligned} w \Vdash P &\iff P \text{ is true at } w, \\ w \not\Vdash \perp, \\ w \Vdash A \wedge B &\iff w \Vdash A \text{ and } w \Vdash B, \\ w \Vdash A \vee B &\iff w \Vdash A \text{ or } w \Vdash B, \\ w \Vdash A \rightarrow B &\iff \forall v \geq w, v \Vdash A \implies v \Vdash B. \end{aligned}$$

Checking implication at the current world only would reduce the semantics to something essentially classical and would be wrong for this purpose.

The certificate is accepted exactly when the order and persistence conditions hold, every assumption in Γ is forced at the root, and G is not forced there. The checker recomputes forcing; any solver-provided table of formula truth values is unnecessary and untrusted.

Lemma 6.1 (Persistence). *If $w \leq v$ and $w \Vdash A$, then $v \Vdash A$.*

Proof. Induct on A . Atoms use valuation persistence, and $\perp$ is never forced. Conjunction and disjunction use the induction hypotheses. For implication, let $t \geq v$. Transitivity gives $t \geq w$, so the implication forced at w applies to t as required. $\square$

Theorem 6.2 (Countermodel soundness). *An accepted model for Γ, G rules out a derivation $\Gamma \vdash_{\text{IPC}} G$ in intuitionistic natural deduction.*

Proof. Prove by induction on a derivation that every world forcing the derivation's assumptions forces its conclusion. Assumptions, conjunction rules, and disjunction introductions are immediate from the forcing clauses. Disjunction elimination uses the forced disjunct at the current world and the induction hypothesis for its corresponding branch. Implication elimination applies the forcing clause with $v = w$, using reflexivity. For implication introduction, move to an arbitrary successor world satisfying its antecedent. Persistence carries the undischarged assumptions there; the induction hypothesis for the body establishes the consequent. Falsity elimination is vacuous because no world forces $\perp$. These cases cover the natural-deduction rules. Thus any derivation of G from Γ would force G at the certificate's root, contradicting the checked result. $\square$

This is a mathematical proof of the certificate format. To obtain a Lean receipt, connect the finite evaluator to an existing formalization’s forcing relation and apply its soundness theorem. Completeness is not needed to check an individual countermodel. Existing Lean formalizations should be reused for this metatheory rather than rewritten merely to attach a Forge name [11].

6.3 Why classical tautologies are essential negative tests

For excluded middle $P \vee \neg P$, use two worlds $0 < 1$, with P false at world 0 and true at world 1. The root does not force P , and it does not force $\neg P$ because a successor forces P . Hence it does not force their disjunction.

The same chain refutes double-negation elimination $\neg\neg P \rightarrow P$. At the root, every successor forcing $\neg P$ would have to force $\perp$, but no successor forces $\neg P$; thus $\neg\neg P$ holds while P does not. Taking Q false everywhere also refutes Peirce’s formula $((P \rightarrow Q) \rightarrow P) \rightarrow P$.

All three are classically valid. Consequently an interface that turns these countermodels into proofs of the *negated Lean formulas* is visibly unsound. These examples are not ornamental: they enforce the worker’s semantic ceiling.

6.4 A branching model catches what chains miss

Let the root have two incomparable terminal successors. Make P true only on the left successor and Q true only on the right. At the root neither $P \rightarrow Q$ nor $Q \rightarrow P$ holds. Thus

$$(P \rightarrow Q) \vee (Q \rightarrow P)$$

is refuted. With one successor making P true and the other keeping it false, the root also refutes $\neg P \vee \neg\neg P$.

The prototype finds both at three worlds. Restricting search to linear chains would miss these examples regardless of its chain-length budget. The implemented proposer enumerates rooted trees, not just chains; the checker accepts arbitrary rooted finite partial orders, including a separately tested diamond.

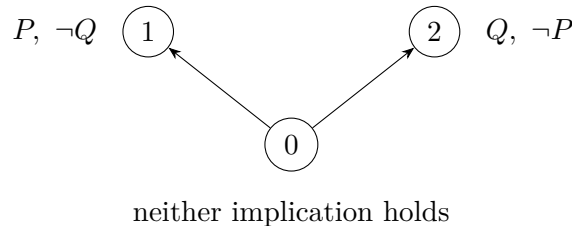


Figure 1: A three-world branching countermodel to propositional linearity. Arrows point toward information extensions; reflexive order edges are omitted.

6.5 The executable bounded proposer

The proposer increases n from one to a configured world bound. For each rooted tree it enumerates atomic upward-closed truth sets, evaluates formulas using bit masks, and proposes a model as soon as the root satisfies the context but not the goal. Nodes are numbered with parents preceding children.

This may enumerate isomorphic trees more than once; no minimality or symmetry optimization is claimed.

Replay is separate: it validates the explicit Boolean relation matrix and uses a cached recursive forcing function that quantifies over all successors. Randomly generated formulas are evaluated through both paths in the test suite. Stored models are replayed without the proposer and without site packages.

When the model budget or world bound is exhausted, the result is UNKNOWN. In particular, the eight familiar IPC-valid schemas in the corpus are not “proved” by absence of a bounded countermodel. They serve as positive controls against an invalid refuter. The fact that IPC has suitable finite models does not turn an arbitrary three-world cap into a decision procedure.

6.6 A concrete scalable SAT backend, proposed but not executed

For a fixed n , introduce relation variables R_{ij} , atomic valuation variables $V_{i,p}$, and formula-value variables $T_{i,A}$ for each subformula. Require reflexivity, transitivity, antisymmetry, a least root, and

$$R_{ij} \wedge V_{i,p} \implies V_{j,p}.$$

Constrain the Boolean connectives by biconditionals, and use

$$T_{i,A \rightarrow B} \iff \bigwedge_{j < n} (\neg R_{ij} \vee \neg T_{j,A} \vee T_{j,B}).$$

Finally assert every $T_{0,A}$ for $A \in \Gamma$ and $\neg T_{0,G}$. Standard Tseitin variables turn the conjunctions into clauses without exponential formula expansion. This encoding has $O(n^3)$ transitivity constraints and $O(n^2s)$ implication work for s subformulas.

A SAT library such as PySAT is a replaceable *model proposer*; its satisfying assignment is decoded into the existing certificate and independently checked [17]. An UNSAT answer for a fixed n carries no unbounded noninhabitation or validity authority in this design. That is true even when the SAT solver itself is correct.

7 Concrete Lean, Forge, Leant, and Djex integration

7.1 Do not invent a new outer controller

The reviewed Forge runtime sketch already has obligations, scope keys, conditional candidates, and replies named `candidate`, `conditional`, `progress`, and `noResult`. Those remain the right entry points for positive proof construction. They are sketches rather than an implemented `forge` runtime [5].

The new positive worker should return a `ConditionalCandidate` whose premises are the local polynomial identities, initial equalities, and a source-to-machine semantic bridge. Its constructor is the appropriate induction theorem. After those premises have Lean proofs, the ordinary Forge acceptance path may install the result. A Python Boolean is never submitted as that proof.

The Kripke lane should return a separate *diagnostic artifact*. The current reply sketch does not distinguish object-logic nonderivability from a source-goal refutation. Add a nominally separate artifact carrying its exact object formula, atom-origin map, and a proof about a selected derivation type. Do not encode it as a candidate for $\neg G$.

7.2 The smallest useful source-to-machine reifier

The first implementation should accept elaborated, monomorphic folds with fixed products of rational registers and explicit list element types. It can be built through six checkable transformations:

1. Locate an equality between supported folds or their fixed polynomial output projections. Obtain the actual elaborated step functions and initial states, not a reparsed display string.
2. Normalize tuple projections and let-bindings using proved equations; distinguish old-state register expressions from newly computed values.
3. Reify literals, addition, multiplication, subtraction over $\mathbb{Q}$, and natural powers into a typed polynomial AST. Refuse all other arithmetic operations in the first version.
4. Reify complete finite constructor matches as control edges. Every constructor branch must have a corresponding transition; do not interpret a partial pattern match as a total transition system.
5. Build the product machine with one shared payload tuple per paired step. Prove the equality between each original step and evaluation of its reified polynomial update.
6. Instantiate the fold/run correspondence and translate output equality into the demanded polynomial difference. Keep this bridge among the candidate’s explicit proof obligations.

These operations can initially be implemented in ordinary **MetaM** using Lean expression construction and standard elaboration. No undocumented incremental **grind** hook is required for this worker. The arithmetic identities can be discharged in separate local goals, and the completed invariant can then be supplied to **grind** as an ordinary lemma.

This makes the contribution orthogonal to local saturation. The documented **grind** engine remains useful for closing local consequences; it is not being misdescribed as lacking congruence closure, arithmetic, or E-matching [12]. The added capability is choosing and proving a finite invariant that expresses an unbounded execution property.

7.3 The initial Lean proof path: source reconstruction

For each accepted certificate, emit the following obligations over the original register and input binders:

$$P_{q_0}(x_0) = 0, \quad G_q = C_q P_q, \quad P_{q'} \circ F_e = M_e P_q.$$

Use **norm_num** for closed rational initial calculations and **ring** or **ring_nf** for the universal polynomial identities. Then use an ordinary induction theorem over the machine’s reachability or input-list execution.

The package’s **emit_lean.py** performs the local source-emission part. It emits 222 named identity obligations from the 37 positive certificates into **lean/GeneratedIdentities.lean**. The file includes **#print axioms** commands for later auditing. It does not emit or prove a bridge from arbitrary original Lean source, and no compiler was available here to validate those identities. The generated count is a count of *local obligations*, not 222 proved user theorems.

lean/Reachability.lean supplies complete, but uncompiled, proof text for three generic induction bridges: reachable-state invariant preservation, fold invariant preservation, and relational fold preservation. A hand-written delayed-accumulator reconstruction appears in **lean/WorkedFold.lean**. These distinguish a concrete proof path from an abstract promise to “replay in Lean.” Their uncompiled status remains a material limitation.

7.4 A reflected checker is the next optimization, not a prerequisite

For larger instances, prove one theorem of the schematic form

```
checkClosure problem certificate = true ->
  forall state, Reachable problem state -> GoalsHold problem state
```

inside Lean. The data types need a semantic interpretation into the same rational polynomial ring used by the source bridge. Soundness factors into canonical decoding, correctness of sparse operations, the evaluated identity lemma, and the reachability induction theorem.

Mathlib’s multivariate polynomial evaluation infrastructure supplies relevant semantics and homomorphism lemmas [14]. Its representation need not be the fastest executable representation. A compact sparse AST can carry its own denotation into `MvPolynomial`, with a proved correctness theorem for normalization. The external worker can retain any faster internal data structure because Lean checks only the emitted certificate.

Clearing denominators row by row can reduce the rational checker to integer polynomial identities. Record each positive denominator and multiply the whole corresponding equation, rather than round coefficients. Prove the nonzero scalar cancellation once. This is a deterministic representation optimization; it does not alter the certificate’s mathematical claim.

7.5 Universal behavioral assertions in Leant

At the reviewed Leant snapshot, the named `:synth f : TYPE where PROP` route checks the exact proposed candidate and attempts positive and negative proofs with `decide` and bounded simplification. A quantified assertion can remain inconclusive when those methods do not close it [6].

Extend that assertion checker with a new dispatch rule, not a new inhabitant search engine. When the asserted property is a supported universal fold equality, feed the exact candidate/reference pair to the product-machine reifier and observable-closure worker. A fully reconstructed Lean proof makes the behavioral assertion pass. A fully reconstructed concrete source execution can falsify it. A reifier refusal, search cutoff, or unproved semantic bridge leaves the assertion inconclusive.

The candidate’s original exact type and source identity remain unchanged. A successful machine certificate for one candidate cannot be used for another candidate with the same displayed type. This reuses the existing source-owned candidate discipline rather than claiming it as an invention.

7.6 Source-level quantifiers and arithmetic domains

A closed rational initial vector is the prototype’s direct input. Universally quantified rational initial parameters can use the special initialization-edge construction in Section 2. More general parameter telescopes require a source bridge that proves this encoding preserves the quantifier order and variable dependencies.

Positive rational polynomial identities also evaluate in a commutative $\mathbb{Q}$ -algebra. This gives a route to real-valued versions of the same equalities, but the initial implementation must not silently cast an arbitrary source semantics into that setting. A rational trace only becomes a source-level counterexample after its values are well typed and its operations are proved to match the source operations. In particular, negative rational states do not automatically provide counterexamples to

natural-number programs.

7.7 A precise Kripke-to-Lean target

Choose an IPC syntax and derivation system already formalized in Lean. Define a Boolean evaluator for finite models and prove that it agrees with the library’s semantic forcing relation. The desired output theorem is schematically

```
checkCountermodel Gamma goal certificate = true ->
  Not (Nonempty (IPC.Derivation Gamma goal))
```

The inner `IPC.Derivation` is an object-language derivation type; the statement is not `Not (denote goal)`.

An object sequent extracted from Djex’s formula structure needs a faithful map of constructor-tagged disjunctions and conjunction arities, and an exact atom identity table. A successful object-model check does not prove the separate translation-completeness theorem for opaque quantified source types or class dictionaries. Until that theorem is available, report only the named object-logic result. Classical fallback may still prove the user’s source formula.

8 Executed experiments and their limits

8.1 Protocol and retained artifacts

The tests ran under Python 3.13.5 and SymPy 1.14.0 on the Linux environment recorded in `results/environment.json`. The case generator uses seed 20260915. Each of 82 corpus cases was run three times in one process; the certificate bytes, not merely the status, were required to agree between repetitions. Search and independent replay were separately timed. Imports and machine-case construction are outside the per-case search timings.

`results/corpus.json` contains the exact problems, limits, and returned certificates or inconclusive reasons. `measurements.csv` contains one row per case. `trials.json` preserves individual timings and search statistics. These observation files are separate because mathematical inputs reject floating-point JSON values while timing measurements are naturally floating point.

Workload	Cases	Closure accepted	Trace accepted	Unknown
Worked machine cases	18	13	4	1
Affine conjugacies	24	24	0	0
Mutated conjugacies	24	0	24	0
All rational machines	66	37	28	1

Table 2: Exact-machine results. “Accepted” means independent Python replay, not a Lean kernel proof. The true nonlinear-growth case is intentionally unknown.

All 73 certificates replay under

```
python -S replay.py
```

IPC workload	Cases	Countermodel accepted	Unknown
Expected nonderivable sequents	8	8	0
Valid-schema controls	8	0	8

Table 3: The bounded IPC worker has no positive theorem verdict. The eight unknown controls are not counted as proved by this experiment.

with no SymPy, NumPy, SciPy, or Z3 module loaded. The command reports 37 closure receipts, 28 execution-counterexample receipts, 8 IPC-countermodel receipts, and 9 cases carrying no conclusion. The largest certificate is 915 bytes in compact canonical JSON. That number excludes its original problem, metadata, pretty-printing whitespace, and generated Lean source.

8.2 Mechanism ablation, not a tactic comparison

A deliberately narrow comparator, `conserved_target_only`, accepts a single-control-node problem only when every requested target is itself conserved by every edge and initially zero. It does not synthesize additional observations or accept finite control flow. It succeeds on 8 of the 37 valid machine cases, whereas observable closure succeeds on all 37.

This result shows that the test suite distinguishes the new mechanism from that comparator. It is *not* a measured gain over all previous Forge prototypes, which include stronger candidate-generation mechanisms, and it is not a comparison with `grind`, `Aesop`, `nlinarith`, or a theorem prover. The cases were designed to expose specific mechanisms, including control-indexed relations and hidden auxiliary equalities.

8.3 Adversarial and differential checks

The retained pytest run reports **151 passed**. Its parameterized tests include all 82 corpus cases; 36 direct certificate/trace/model mutations; nontransitive and nonmonotone model rejection; invalid numeric encodings; missing edge and goal coverage; wrong problem reuse; control-flow mismatch; empty and unreachable cases; resource cutoffs; and an explicit regression for the seven-sample trap.

Polynomial substitution is tested against SymPy on 200 generated sparse polynomial instances. The checker expands and evaluates with its own rational arithmetic, while the reference computes simultaneous symbolic substitution. Random small formulas and finite frames exercise both bitset and recursive forcing; those loops also call the actual countermodel checker on each root-level result. These tests can expose representation and implementation errors. They do not replace a formal correctness proof of the Python programs.

Every one of the eight named nonderivable IPC cases was found at its expected smallest world count within the enumerated tree family: one, two, or three worlds. This is an observation on these cases, not a general minimal-model algorithm or a promise that a world bound is complete.

8.4 Timing observations are intentionally secondary

The median of the machine cases’ three-run median search times was about 2.0 milliseconds; the corresponding IPC statistic was about 0.045 milliseconds. These are small in-process synthetic instances, with symbolic caches potentially warmed, and are not predictions for Lean reification,

subprocess startup, certificate reconstruction, or kernel checking. The raw per-case values are retained rather than being converted into a headline speedup claim. The complete experiment driver, excluding initial construction of the machine case list, took under a second in this run.

8.5 A failed attempt retained, rather than erased

The first standalone replay failed before checking any mathematics. Timing floats had been placed in the same JSON envelope as exact certificate data; the strict decoder correctly rejected them. The repair separated observation files from the exact corpus. The policy rejecting mathematical floats was *not* relaxed. A diagnostic description is retained at `results/diagnostics/initial-replay-failure.txt`, and the successful replay log is a different file.

This packaging failure is not a rejected mathematical theorem, but recording it prevents a reader from mistaking every historical run for successful evidence.

8.6 What these experiments establish, and what they do not

They establish that the supplied producers and independent replay paths agree on this explicit corpus; that the new closure mechanism handles the stated examples; and that specified corruption and semantic-scope tests reject their invalid inputs. They also provide concrete data for the next Lean integration step.

They do not establish correctness for every input, a production-grade parser, a formally verified checker, a dependent Lean reifier, a compiled Forge tactic, or an advantage on a representative Lean theorem distribution. Neither Lean, Lake, nor Elan was present in the execution environment. The uncompiled Lean files cannot fill that evidentiary gap by their mere existence.

9 Implementation plan, libraries, and acceptance gates

9.1 Concrete library choices

Component	Library or module	Exact role and status
Prototype algebra	SymPy 1.14.0 Poly, QQ; custom span manager	Executed. Proposes exact polynomials and coordinates; never used by standalone replay.
Prototype replay	Python Fraction; checker.py	Executed. Canonical sparse arithmetic, complete edge coverage, original-target checks.
Larger rational systems	SymPy DomainMatrix.rref_den; FLINT fmpq_mat_rref	Proposed search optimization. Both have documented exact-linear-algebra interfaces; emitted coordinates must still be multiplied back exactly [15, 16].
Lean identity semantics	Mathlib MvPolynomial; ring, norm_num	Proposed/partly source-emitted. Local identities can be reconstructed before investing in reflection [14].
Positive IPC search	Existing itauto or Djex LJT	Reuse; not implemented again here [13, 7].

Component	Library or module	Exact role and status
Finite-model search	Included rooted-tree enumerator	Executed. Small bounded proposer with independent forcing replay.
Larger finite models	PySAT solver interface	Proposed only. Produces a model for the same checker; UNSAT at a finite bound is not a proof of validity [17].
Lean IPC metatheory	Existing Kripke soundness formalization	Proposed reuse. Connect finite Boolean forcing to the selected library’s semantics and derivation type [11].

FLINT and `DomainMatrix` are not runtime dependencies of the supplied checker, and PySAT was not installed or run. Their entries are concrete next implementations, not hidden explanations of the observed timings. A production integration must pin actual compatible library revisions; current documentation is not a compatibility guarantee for Forge’s pinned Lean toolchain.

9.2 Suggested module additions

The following names are proposed additions, not claims about files already present in Forge:

```
Forge/Worker/Observable/IR.lean
Forge/Worker/Observable/ReifyFold.lean
Forge/Worker/Observable/Certificate.lean
Forge/Worker/Observable/Reconstruct.lean
Forge/Worker/Observable/Reachability.lean
Forge/Diagnostic/IPC/FiniteModel.lean
Forge/Diagnostic/IPC/Check.lean
Forge/Diagnostic/IPC/SoundnessBridge.lean
```

Keep the source-to-machine bridge in a separate module from certificate checking. A correct polynomial checker attached to an incorrect translation is not a proof of the user’s theorem. Similarly, separate the IPC object formula and its atom origins from any source-level completeness claim.

9.3 Gate 1: the first actual Lean milestone

Use the Lean and Mathlib pair recorded by the reviewed Forge baseline as the initial target, then resolve any real API differences during compilation. First compile `Reachability.lean` and all 222 generated identity obligations in a project using that pinned Mathlib revision. Preserve compiler output and transitive axiom reports. No source file containing `sorry` or a placeholder axiom is an acceptable substitute for this gate.

Next construct *one complete* machine-soundness theorem by combining the identities with the induction bridge, and prove the original delayed-fold statement from it. A compilation of only the local polynomial identities does not close this milestone. Its negative counterpart must produce and check the actual length-two source execution, not merely fail to prove the positive statement.

The supplied Lean files contain no `sorry` or added axioms, but their textual absence is only a preflight property. Compilation and a positive kernel acceptance signal remain necessary, under Forge’s existing acceptance policy.

9.4 Gate 2: a real reifier with exact-source controls

Implement only the monomorphic tuple/list fragment first. Require each supported expression to carry a proved denotation equation. Test constants, negative coefficients, rational denominators, old-state projections, powers, multiple payloads, and finite control branches. Reject unsupported operations rather than silently abstracting them away.

Use pairs of examples differing in one material source detail: simultaneous versus sequential updates, rational versus natural subtraction, one input versus two independent inputs, a missing constructor case, or a different initial parameter dependency. Reuse of a certificate across these pairs must fail unless a new semantic bridge is proved. The anti-substitution principle is particularly important for the product construction: sharing one payload is what states the intended equivalence theorem.

9.5 Gate 3: universal behavioral acceptance in Leant

Add the dispatcher described in Section 7. At unchanged candidate-search limits, require one exact candidate to satisfy an unbounded fold assertion that the existing bounded simplification route leaves open. Retain the candidate’s type replay, the complete behavioral proof, and the machine certificate. Also retain a type-correct wrong candidate with a concrete source counterexample, and a supported-but-truncated case with an inconclusive verdict.

Finding a different candidate at a much larger synthesis budget may be useful, but it does not demonstrate that this new verifier closes the original bounded behavioral query. Keep synthesis coverage and proof-checking capability as separate measurements.

9.6 Gate 4: IPC diagnostics that cannot refute classical Lean

Connect the finite checker to the chosen Lean IPC soundness theorem. Require Lean proofs of object-level nonderivability for the eight stored countermodels, including excluded middle and Peirce’s formula. In the same project, allow the corresponding classical source formulas to be proved normally. The coexistence of those results is a correctness test, not a contradiction.

Reject a changed atom map, a missing contextual hypothesis, a nonmonotone valuation, and a changed object calculus. A timeout in model search must leave UNKNOWN. This gate does not assert a full translation-completeness theorem for Djex’s higher-rank or dictionary-bearing source language.

9.7 Gate 5: evaluate real capability, then optimize

Build a held-out collection of list folds, small recurrences, stream summaries, and equivalent accumulator representations from actual Lean developments. Keep source files and target theorems fixed before tuning. Compare at least: ordinary baseline tactics, the same tactics with a hand-supplied invariant, and Forge with automatic observable closure. This isolates the value of invariant discovery from arithmetic reconstruction.

Measure reification time, search time, emitted identity count, certificate bytes, proof-term size, kernel time, peak memory, and solved-source-theorem count. Keep malformed-input refusals and budget exhaustion out of solved counts. Only such a benchmark can support a claim that the new *Lean* worker is practically more capable than its baseline.

9.8 Optimizations that preserve the certificate language

Sparse and fraction-free elimination. The current span manager is small and readable. For larger spaces, use sparse monomial columns and fraction-free row operations; retain original-generator coordinate witnesses. Check candidate dependencies exactly even when modular arithmetic proposes a rank or pivot structure.

Strongly connected components. Condense the control graph. Goals in successor components can propagate backward in reverse topological order; only cyclic components need a local fixed point. This avoids revisiting an acyclic prefix after its downstream spaces have stabilized. The certificate still lists every original edge.

Proof-aware basis changes. A different basis can make M_e sparser or reduce coefficient size. Supply a rational change-of-basis matrix and check it by multiplication, then emit identities in the cheaper basis. Do not select a numerically approximate basis or remove an observation merely because its sampled values appear dependent.

Resource-aware counterexamples. The degree grid is complete locally but can be large in many payload coordinates. Try simple exact points first, then use the proved grid as a fallback. A SAT/SMT proposal can accelerate witness choice, but the final execution trace must still replay in the original source semantics.

These are implementation options, not measured improvements in the current package. The simple baseline was retained so that the certificate mechanism is auditable and its failures are easy to reproduce.

10 Research extensions and final recommendation

10.1 Where the next mathematical work should go

Guarded closure modulo proved constraints. If an edge has a polynomial equality guard $h(x, u) = 0$, preservation only needs to hold on that variety. A certificate could replace (7) by a sum of invariant generators and explicit guard-ideal multiples. Search then needs a carefully chosen quotient representation or a separate ideal-membership proposer. Inequality guards require a different justification. This extension is not implemented, and dropping a guard invalidates source-level counterexamples.

Combine finite vector spaces with invariant ideals. The true $x \mapsto x^2$ example suggests a measured hybrid: attempt observable closure, then use the discovered growing monomials as candidates for the repository’s existing polynomial-multiplier worker. The multiplier worker must independently prove ideal preservation. The closure failure is a proposal signal, not a proof that the resulting ideal is invariant.

Input alignment and simulations. The current product construction is synchronous. Stuttering or multi-element-consuming programs require an explicit simulation relation and a measure ensuring the intended input alignment. A finite-control alignment automaton may cover useful fragments, but a certificate must state which input prefixes each component has consumed.

Certificate-producing constructive synthesis. A negative Kripke model can be paired with a positive LJ proof search in a two-sided service. Positive proof terms go through the existing source re-elaboration boundary. Negative models go through object-logic soundness and an independently justified source translation gate. This gives useful answers on both sides without granting heuristic

failure the status of semantic evidence.

10.2 What should be merged first

The highest-value first merge is the observable machine IR, exact closure producer, sparse replay checker, and the explicit corpus. They provide a small, coherent subsystem with a stated completeness theorem and real negative witnesses. The next merge should compile the generated identities and complete the source-to-fold semantic bridge. An outer **forge** wrapper can then exploit the new lemma as an ordinary checked fact.

The Kripke checker is a useful parallel contribution, provided its result type is explicitly about IPC derivability. It should not be used to strengthen a source-level noninhabitation claim before a faithful translation theorem has been supplied.

10.3 The substantive change in capability

The algebraic worker changes the question from “*which invariant template should I guess?*” to “*which finite space of observations must be zero for this requested output to remain zero?*” In a broad state-affine fragment that question is answered by exact finite-dimensional algebra, and a negative answer has a concrete input word. The logical worker changes a bare failed proof search into a separately inspectable model, while carefully stating which proof language the model excludes.

Neither contribution is a new foundation for mathematics or a replacement for Lean’s kernel. Together they are a concrete next step for Forge: additional proof and diagnostic capabilities, small certificate languages, executed prototypes, and explicit integration gates rather than another general-purpose controller proposal.

A Certificate formats and executable entry points

A.1 Canonical rational polynomials

A rational is encoded as `[numerator, denominator]`, with positive denominator and relatively prime integers. Booleans are not accepted as integers, even though Python’s type hierarchy normally allows that coercion. A polynomial is a lexicographically sorted list of nonzero terms

```
[ [exponent_vector, [numerator, denominator]], ... ]
```

with unique monomials. The empty list is zero. Exponent vectors have the externally specified arity. The decoder rejects duplicate JSON keys, floating-point values, unreduced rationals, explicit zero coefficients, and unknown fields at mathematical boundaries.

The current implementation limits raw polynomial lists to 2,048 terms, encoded total degree to 64, and rational numerator/denominator sizes to 4,096 bits. Intermediate sparse arithmetic has a separate 20,000-term cap. A machine has at most 16 state variables, 3 payload variables, 16 control nodes, and 64 edges; a certificate basis has at most 128 entries per node. Default search limits are smaller in several dimensions. These engineering limits are not part of the mathematical completeness theorem.

These checks are not a hardened sandbox. Symbolic expansion, JSON nesting, big-integer work, and adversarial expression sizes still require process-level memory/time isolation before use on untrusted

external inputs.

A.2 Machine and positive certificate

The source problem fixes its state and input dimensions, initial node and state, all original edges and updates, and output goals at every control node. Its semantics tag is exactly `rational-unrestricted-input`. The certificate cannot change those choices.

```
{
  "kind": "observable_closure",
  "basis": [ /* one polynomial list per node */ ],
  "goal_coordinates": [ /* every original goal */ ],
  "edge_coordinates": [ /* every original edge, in original order */ ]
}
```

Each edge row lists payload exponent vectors and rational coordinate vectors in the source basis. The checker reconstructs the whole polynomial identity. Omitting a row, edge, or goal does not weaken the task: it makes the certificate invalid. Basis independence is deliberately not required for soundness.

A.3 Execution certificate

```
{
  "kind": "execution_counterexample",
  "steps": [ {"edge": 0, "input": [[0,1]]}, ... ],
  "goal_index": 0
}
```

Replay starts from the original initial state. It checks edge applicability by control node and evaluates all updates simultaneously. The indexed final goal must be nonzero. There is no trusted final-state field, no user-supplied residual value, and no need to trust the producer's invariant search.

A.4 IPC problem and countermodel

The problem has fields `version`, `kind`, `logic`, `context`, and `goal`, with `logic = "IPC"`. Formula nodes are `["var", name]`, `["bot"]`, or a three-entry list beginning with `"and"`, `"or"`, or `"imp"`.

```
{
  "kind": "finite_kripke_countermodel",
  "worlds": 2,
  "root": 0,
  "relation": [[true,true],[false,true]],
  "valuation": {"P": [false,true]}
}
```

This particular model refutes excluded middle as an IPC formula. The checker accepts at most 32 worlds; the recorded proposer uses at most 3. A changed logic tag is rejected instead of being interpreted as a compatible proof system.

A.5 Reproduce the experiment

From the archive’s root directory:

```
python -m pip install -r requirements.txt
python -m pytest -q
python -S replay.py
python -m forge_delta.experiment --output reproduced-results --repeats 3
python emit_lean.py
```

The experiment refuses to overwrite a nonempty output directory. Its timings will vary across machines; its deterministic problems and accepted certificate contents should not. The full corpus is available directly to callers that prefer the Python API:

```
from forge_delta.cases import machine_cases
from forge_delta.search import solve
from forge_delta.checker import verify_positive

case = machine_cases()[0]
result = solve(case["problem"], case["limits"])
assert result["status"] == "proved"
assert verify_positive(case["problem"], result["certificate"])
```

Here “proved” is the prototype’s machine-property status. It does not mean that a Lean proof was constructed. For portable standalone replay use the explicit checker rather than interpreting status strings as authority.

A.6 Build the article and attempt the Lean gates

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error forge_delta.tex
pdflatex -interaction=nonstopmode -halt-on-error forge_delta.tex
```

The second pass resolves cross-references and the table of contents. The archive includes all article source sections. It does not redistribute fonts or third-party repositories.

For Lean, use an existing project with the intended pinned Mathlib revision, then run `lake env lean` on each supplied file using its absolute path. The repository’s recorded target was Lean v4.34.0 with Mathlib 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa. This package did not download, compile, or certify that toolchain. The provided instructions do not silently install dependencies or assert that generated files already pass.

B Source audit, scope of originality, and artifact ledger

B.1 Pinned repository context

Repository	Reviewed revision and principal material
Forge	674521027d968d59f7b83220ed52304a85cb55e2 README; recursive tree; <code>article/sections/05-structural.tex</code> ; <code>article/sections/B-provenance.tex</code> ; <code>docs/STATUS.md</code> ; <code>docs/IDEAS-FROM-LEANT-DJEX.md</code> ; <code>lean/Forge/Design/Runtime.lean</code> .
Leant	6bf05ad78c467989e68290f2d08bbbed40802d485 Tree and README context; pinned <code>docs/behavioral-synthesis.md</code> .
Djex	e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5 Root and Djinn trees; pinned <code>djinn/src-core/Djinn/Internal/LJTFormula.hs</code> .

The audit focused on the merged design and the code directly relevant to the proposed integration. It was not a line-by-line review of all nine frozen upstream submissions or every Leant/Djex source file. An incomplete code-search response was not treated as proof that a term or idea is absent from the repository. The claim of increment is therefore grounded in the specific baseline passages identified in Section 1, not in a universal claim of priority over all existing literature or files.

The neighboring repositories are evidence of actual interfaces and existing behavioral-verification mechanisms. Their test counts are not added to this package’s counts, and their source text is not redistributed in the archive.

B.2 Contribution and reuse ledger

Item	Attribution and status
Certificate-producing outer planner	Already in Forge. Reused as context only.
Invariant and observable spaces	Established mathematics. The exact control-indexed coefficient closure is developed here as an incremental worker, with its fragment theorem and concrete formats.
Rational sparse replay	New code in this package, standard algebra. Executed; not formally verified.
Path-origin trace extraction	Implemented here for the stated unrestricted-input machine IR. The counterexample is checked independently of its provenance.
Kripke semantics and IPC soundness	Established logic. A finite model proposer/checker and integration discipline are implemented/proposed here; existing Lean metatheory should be reused.
Generic validation, source identity, trust policy	Already treated extensively by Forge and its neighboring projects. Not claimed as new.

Item	Attribution and status
Universal behavioral fold dispatcher	Concrete proposed extension to the reviewed Leant assertion checker; not integrated into Leant.
Lean reconstruction	222 generated local obligations, three generic induction bridges, and a worked example. All uncompiled here.

B.3 Archive contents

Path	Purpose
article/forge_delta.tex, article/sections/ article/forge_delta.pdf	Main article and complete modular source. Rendered article.
forge_delta/search.py	Exact SymPy closure, retained-origin witnesses, narrow ablation.
forge_delta/checker.py	Standard-library rational-machine replay and strict mathematical decoder.
forge_delta/kripke.py	Bounded model proposer and separate forcing replay.
forge_delta/cases.py	Transparent worked and seeded case generators.
forge_delta/experiment.py	Non-overwriting reproducible experiment runner.
tests/test_delta.py	151-test suite, including mutation and differential tests.
results/	Exact corpus, observations, summaries, logs, generated-Lean count, and an initial packaging failure record.
replay.py, emit_lean.py	Site-package-free replay and local Lean identity emission.
lean/	Uncompiled integration targets and explicit status notes.
docs/	Source audit, implementation limits, and merge-oriented notes.

B.4 Remaining acceptance obligations

The prototype is sufficient to reproduce the mechanism, inspect its exact proof certificates, and develop the next implementation. It is not sufficient to claim that Forge now proves additional Lean theorems. That claim still requires the pinned Lean compilation, denotation bridge, completed induction composition, and representative source-goal evaluation specified in Section 9. These are explicit next engineering gates, not missing explanations concealed behind a test-pass count.

References

- [1] Forge project. *Forge repository*, reviewed revision (see Appendix B). <https://github.com/VladimirReshetnikov/Forge/tree/674521027d968d59f7b83220ed52304a85cb55e2>.
- [2] Forge. *Structural-search section*, merged article, Section 5, at the reviewed revision. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/05-structural.tex>.

- [3] Forge. *Implementation status*, reviewed revision. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/docs/STATUS.md>.
- [4] Forge. *Ideas from Leant and Djex*, reviewed revision. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [5] Forge. *Runtime and scheduler contract sketch*, reviewed revision. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/lean/Forge/Design/Runtime.lean>.
- [6] Leant. *Behavioral assertions during synthesis*, reviewed revision. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbcd40802d485/docs/behavioral-synthesis.md>.
- [7] Djex. *Djinn.Internal.LJTFormula*, reviewed revision. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/djinn/src-core/Djinn/Internal/LJTFormula.hs>.
- [8] Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6, 133–151, 1976. <https://doi.org/10.1007/BF00268497>.
- [9] Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control. *PLOS ONE*, 11(2):e0150171, 2016. <https://doi.org/10.1371/journal.pone.0150171>.
- [10] Yu-Fang Chen, Ondřej Lengál, Tony Tan, and Zhilin Wu. *Register automata with linear arithmetic*. arXiv:1704.03972v2, 2017. <https://arxiv.org/abs/1704.03972>.
- [11] Dafina Trufaș. Intuitionistic propositional logic in Lean. *EPTCS*, 410, 133–149, 2024. <https://doi.org/10.4204/EPTCS.410.9>; <https://arxiv.org/abs/2410.23765>.
- [12] Lean Language Reference. *The grind tactic*. Online reference consulted September 15, 2026. The latest URL is mutable. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [13] Mathlib documentation. *Mathlib.Tactic.ITauto*. Consulted September 15, 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/ITauto.html.
- [14] Mathlib documentation. *Mathlib.Algebra.MvPolynomial.Eval*. Consulted September 15, 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/MvPolynomial/Eval.html.
- [15] SymPy documentation. *Introducing the domainmatrix of the poly module*. Version 1.14.0 documentation, consulted September 15, 2026. <https://docs.sympy.org/latest/modules/polys/domainmatrix.html>.
- [16] FLINT documentation. *fmpq_mat.h—matrices over the rational numbers*. Consulted September 15, 2026; the online development documentation is mutable. https://flintlib.org/doc/fmpq_mat.html.
- [17] PySAT documentation. *SAT solvers’ API (pysat.solvers)*. Consulted September 15, 2026; proposed dependency, not used in these experiments. <https://pysathq.github.io/docs/html/api/solvers.html>.